

2026_Neural Network_ Lecture 2 notes

Lecture 2 — Making it work: optimizers, regularization, convolutions, attention

Lecture 1 built the machine. This lecture is about the choices that decide whether it learns anything useful:

1. How to walk through a landscape with a million dimensions (optimizers)
2. How to stop a network from memorizing (regularization)
3. How to put what you know about the data into the architecture (convolutions)
4. How to let the network decide what to compare with what (attention, transformers)
5. What generative models are, in one table

Lab 2 follows the same order on synthetic spectra.

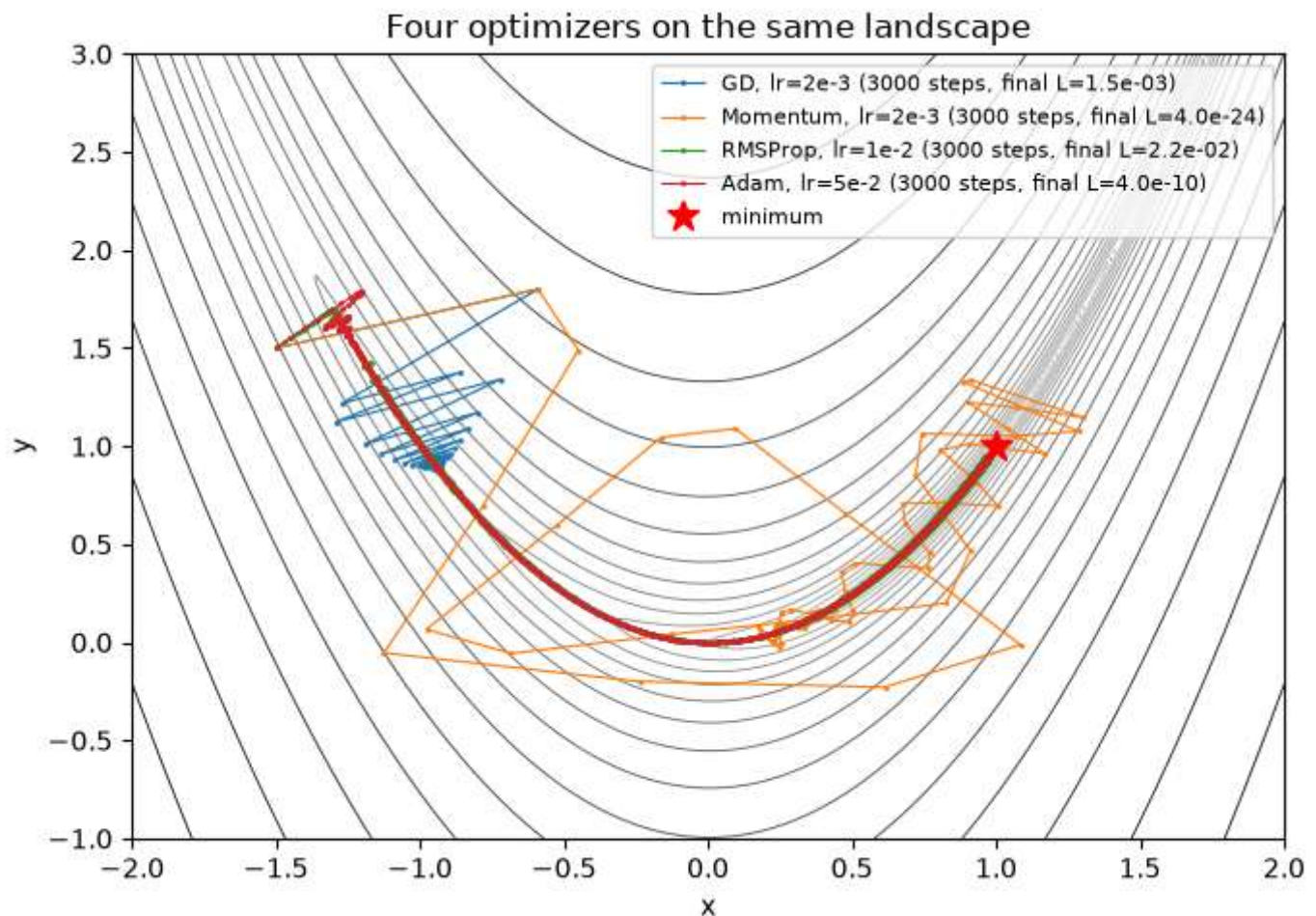
Review: one training step

Step	Formula	Lab 1 code
Forward	$\mathbf{a}^{(l)} = h(W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$	<code>A1 = sigmoid(W1 @ X + b1)</code>
Loss	$L = -\frac{1}{n} \sum_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$	<code>-np.mean(Y*np.log(A2) + ...)</code>
Backward	$\delta^{(L)} = \hat{\mathbf{y}} - \mathbf{y}, \quad \delta^{(l)} = h'(\mathbf{a}^{(l)}) \odot W^{(l+1)\top} \delta^{(l+1)}$	<code>dZ1 = (W2.T @ dZ2) * A1*(1-A1)</code>
Gradient	$\partial L / \partial W^{(l)} = \delta^{(l)} \mathbf{a}^{(l-1)\top} / n$	<code>dW1 = dZ1 @ X.T / m</code>
Update	$W \leftarrow W - \eta \partial L / \partial W$	<code>W1 -= learning_rate * dW1</code>

Everything in this lecture changes only the last line, or the data that go into the first.

The loss landscape, and why we cannot use Levenberg–Marquardt

- Curve fitting (`scipy.optimize.least_squares`, KiMoPack) uses the Jacobian $J_{ij} = \partial r_i / \partial p_j$ and the Gauss–Newton Hessian $H \approx J^\top J$. Curvature turns a valley into a direct route: a few dozen evaluations
 - For P parameters $J^\top J$ has P^2 entries. With $P = 10^6$ that is 10^{12} numbers (4 TB) and a P^3 solve. Impossible
 - Gradient descent needs only $\partial L / \partial w$, which back-propagation gives at linear cost in P . The price: no curvature information
 - Network landscapes are ill-conditioned (stiff in some directions, flat in others), non-convex, and have many equivalent minima. Plain gradient descent has a learning-rate window of about one decade: below it crawls along the valley, above it bounces across it and diverges



Optimizer trajectories on the Rosenbrock function

Lab 2, Part 1: four optimizers on the Rosenbrock valley; `least_squares` for comparison.

Batch, stochastic and mini-batch gradient descent

Question: when and how to update parameters:

Variant	Gradient computed on and Parameters updated after	Per step	Character
Batch GD	all n training examples (one epoch)	expensive	exact gradient, smooth path
Stochastic GD	one example	cheap	very noisy
Mini-batch SGD	$B = 32 \dots 512$ examples, several updates per epoch	one matrix product	noisy enough to leave bad minima, cheap enough to take many steps

- Everyone uses mini-batches; "SGD" in practice means mini-batch
- The noise is a feature: it helps to escape saddle points and sharp minima, and it acts as a regularizer. Deep-network loss surfaces contain flat regions, narrow valleys, saddle points, and many nearly equivalent solutions. A full-data gradient can be very smooth and may move slowly in such regions. Mini-batch adds a small random "kick".
- Batch size trades gradient quality against the number of steps you can afford; larger batches usually want larger learning rates

Lab 2, Part 3: `batch_size=32` in `train()`.

Momentum

Have a small learning step, but consider the previous updates to speed up (take larger training steps in the same direction if merited)

$$v_t = \beta v_{t-1} + g_t, \quad w_{t+1} = w_t - \eta v_t \quad (\beta \approx 0.9)$$

- A heavy ball rolling downhill: the velocity averages the last $\approx 1/(1 - \beta) = 10$ gradients
- Components that flip sign at every step (across the valley) cancel; components that keep their sign (along the valley) add up to $g(1 - \beta)$, ten times the plain step
- Nesterov momentum evaluates the gradient at the look-ahead point $w_t - \eta \beta v_{t-1}$; a small but consistent improvement
- PyTorch: `torch.optim.SGD(params, lr, momentum=0.9, nesterov=True)`

Lab 2, Task 1.1 and Task 2.1.

RMSProp: a learning rate per parameter

Idea: If a parameter repeatedly has large gradients, take smaller steps in that parameter. If it has consistently small gradients, allow relatively larger steps.

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2, \quad w_{t+1} = w_t - \eta \frac{g_t}{\sqrt{s_t} + \epsilon}$$

- s_t is the running mean of the squared gradient, per parameter
- Dividing by $\sqrt{s_t}$ makes every parameter move by about η per step, whatever the scale of its gradient: stiff directions get small steps, flat directions large ones
- This is a cheap, diagonal stand-in for the curvature we could not afford
- Cures the vanishing/exploding scale problem between layers; the first layer of a deep net often has gradients 10^3 times smaller than the last

Adam: momentum and RMSProp combined

	Formula	Meaning
1. First moment.	$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$	running mean of the gradient (momentum), $\beta_1 = 0.9$
2. Second moment	$s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2$	running mean of the squared gradient (RMSProp), $\beta_2 = 0.999$
3. Bias correction	$\hat{m}_t = m_t / (1 - \beta_1^t),$ $\hat{s}_t = s_t / (1 - \beta_2^t)$	both averages start at zero and would be too small for the first $\sim 1/(1 - \beta)$ steps
4. Update	$w_{t+1} = w_t - \eta \hat{m}_t / (\sqrt{\hat{s}_t} + \epsilon)$	$\eta = 10^{-3}$ is a good default, $\epsilon = 10^{-8}$

- The default optimizer since 2015 (Kingma & Ba). Tolerates a 10–25× larger learning rate than plain gradient descent and needs little tuning. Its step is about η per parameter whatever the gradient, so near the minimum it hovers until a schedule lowers η
- **AdamW** applies weight decay directly to w instead of adding it to g ; use it when you regularize (next slides). `torch.optim.AdamW`
- SGD with momentum still wins some image benchmarks when tuned carefully; Adam wins on time-to-a-good-result

Lab 2, Task 1.2: Adam in ten lines of numpy.

Learning-rate schedules

- The learning rate is the most important hyperparameter. Too small: nothing happens. Too large: the loss oscillates or becomes `nan`
- A **schedule** lowers it during training: large steps to cross the landscape, small steps to settle in. AdamW is adaptive, but it is not self-annealing in the usual sense and thus the settling phase is difficult. A scheduled learning rate fixes that.

Schedule	Rule	Where you met it
Exponential decay	$\eta_t = \eta_0 \gamma^t$	Lab 1: <code>learning_rate *= 0.999</code> per iteration
Step decay	$\eta = \eta_0 \gamma^{\lfloor \text{epoch}/s \rfloor}$	classic image networks
Cosine annealing	$\eta_t = \frac{\eta_0}{2} \left(1 + \cos \frac{\pi t}{T} \right)$	most current papers
Warm-up	linear increase over the first few hundred steps	transformers: Adam's $\hat{s}_t$ is unreliable at the start
Reduce on plateau	divide by 10 when the validation loss stalls	<code>keras.callbacks.ReduceLROnPlateau</code>

- First tune η_0 with a quick scan over $10^{-4} \dots 10^{-1}$; then add a schedule

Lab 2, Task 1.3 (learning-rate scan) and the cosine example in Part 2.

Initialization: where the journey starts

- Remember: it is a fitting procedure. Starting weights of the wrong size make the signal shrink or explode from layer to layer, and the gradient with it
- Goal: keep the variance of the activations constant through the layers

Rule	Weights drawn with variance	For
Xavier / Glorot (2010)	$\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}$	tanh, sigmoid
He / Kaiming (2015)	$\sigma^2 = \frac{2}{n_{\text{in}}}$	ReLU (half of the units are off, so twice the variance)

- numpy: `W = np.random.randn(n_out, n_in) * np.sqrt(2 / n_in)`

- PyTorch and Keras do this for you; you only notice when you override it
- All-zero or all-equal initialization is fatal: every unit in a layer stays identical forever (symmetry is never broken)

Lab 2, Task 2.3: weights with $\sigma = 0.01$ stall the training.

Vanishing and exploding gradients, and the three cures

- During back-propagation, the error signal must travel from the output layer back to the early layers $\delta^{(l)} = h'(\mathbf{a}^{(l)}) \odot W^{(l+1)\top} \delta^{(l+1)}$: every layer multiplies the error by h' and by $W^\top$.
Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$, derivative = $\sigma(z)(1 - \sigma(z))$, Twenty layers of sigmoid ($h' \leq 1/4$) shrink it by 4^{-20}
- Cures, all in daily use:
 1. use **ReLU** instead of sigmoid/tanh: $h' = 1$ for active units
 2. **Batch normalization** (Ioffe & Szegedy 2015): normalize each layer's pre-activations over the mini-batch to zero mean and unit variance, then let the network learn a scale and a shift. Every layer sees well-scaled inputs, whatever the layers before it do. This happens during the *Forward* propagation. $X \rightarrow Z = XW^\top + b \rightarrow \text{BatchNorm}(Z) \rightarrow \text{ReLU} \rightarrow A$.
 3. **Skip (residual) connections** (He et al. 2015, ResNet): $\mathbf{a}^{(l+1)} = \mathbf{a}^{(l)} + f(\mathbf{a}^{(l)})$. The gradient has a path of slope one straight through. This is what made 100-layer networks trainable, and every transformer has them



└ Conv → BatchNorm → ReLU → Conv → BatchNorm — (+) → ReLU → output

- Sometimes, Gradient clipping (cap the norm of g) stops explosions in recurrent networks

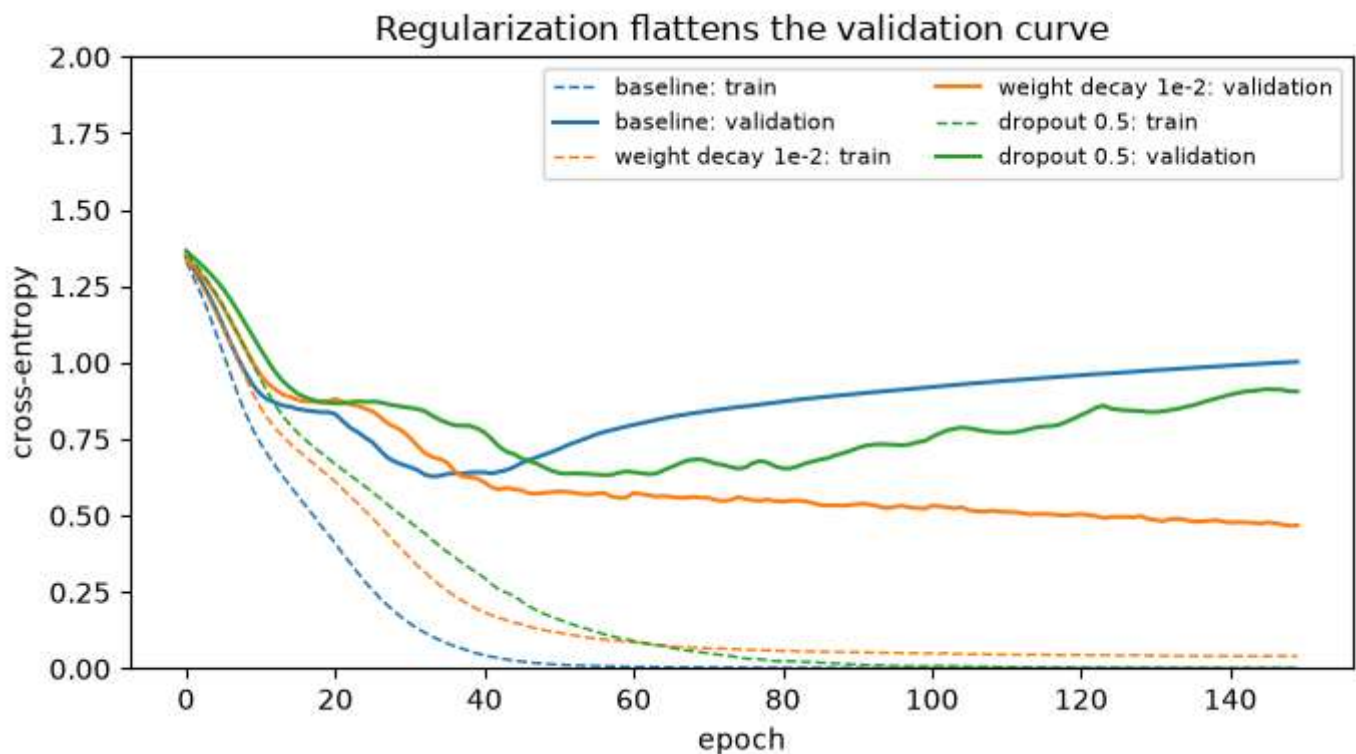
Regularization I: limit what the network can memorize

- A network with more parameters than training examples can fit anything, including the noise. Regularization limits the fit
- **Early stopping**: stop when the validation loss stops improving; restore the best weights. The cheapest cure, (should always be on!!!)
- L_2 **penalty / weight decay**: $L' = L + \lambda \sum w^2$, gradient $+2\lambda w$, so each step also shrinks every weight by $(1 - 2\eta\lambda)$. Small weights mean a smoother function — think a low-order polynomial
- L_1 **penalty**: $L' = L + \lambda \sum |w|$, drives weights to exactly zero: a sparse network, feature selection
- Both are the Bayesian prior on the weights that you know from ridge and lasso regression

Lab 2, Task 3.1 and 3.2, on 60 spectra with a 10^5 -parameter MLP.

Regularization II: change what the network sees

- **Dropout** (Srivastava et al. 2014): during training, set each hidden unit to zero with probability p (0.2–0.5); at test time use all units, scaled. No unit can rely on a specific partner, so the network becomes an ensemble of thinned networks. Remember `model.train()` / `model.eval()`: dropout must be off when you evaluate
 - **Batch normalization** also regularizes, because each example is normalized with different batch mates
 - **Data augmentation**: create new training examples with transformations that do not change the label
 - images: flips, crops, rotations, colour jitter
 - spectra: shift the calibration by a few channels, add noise, tilt the baseline, scale the amplitude
 - Augmentation is the only method that adds *knowledge* about the data. On drifting spectra it beats the other three by a wide margin



97f6c261e3eba5153180aa1b24e8991f.png

Validation curves for baseline, weight decay, dropout and augmentation

Hyperparameter tuning

- Hyperparameters are set by you, not learned: **learning rate**, **batch size**, **depth**, **width**, **dropout**, **weight decay**, **schedule**, **architecture**
 - Order of importance:
 - learning rate first,
 - then architecture size,
 - then regularization,
 - then everything else
-

Hyperparameter 2

The tuning of the hyperparameters can be automated. So you test on a smaller training batch the behavior using:

- **Grid search**: all combinations of predefined values. Systematic; cost grows as k^d
 - **Random search** (Bergstra & Bengio 2012): sample at random. Better than grid for the same budget, because few hyperparameters matter and grid wastes points on the ones that do not
 - **Bayesian optimization** (Optuna, Ax): fit a surrogate model to past trials, propose the next one. Best for expensive runs
 - **Hyperband / successive halving**: start many runs, kill the worst early
 - Always tune on the validation set, never on the test set. Report the test score once
-

Debugging a network that does not learn

1. Check that the initial loss is close to pure random: $\ln m$ for m balanced classes ($\ln 2 = 0.69$, $\ln 4 = 1.39$, $\ln 10 = 2.30$). If not, the output layer or the labels are wrong.
2. Overfit a single mini-batch: the training loss must go to zero. If not, the model or the gradient flow is broken
3. Scan the learning rate over four decades; look for `nan` (too large) and a flat line (too small)
4. Check the input scaling: features of order one, no feature 10^3 times larger than the rest
5. Look at the gradient norms per layer: they should differ by less than a factor of 100

- Plot training and validation loss every epoch. Always. Overfitting, plateaus and divergence are all visible there
- Fix the random seeds and compare two runs. If the seed changes your conclusion, you have no conclusion

Lab 2 does 1, 3, 5 and 6 explicitly.

Architecture as inductive bias

Every architecture encodes an assumption about the data. The right assumption buys data efficiency; the wrong one costs accuracy.

Architecture	Assumption built in	Typical data	Parameters
Dense (MLP)	none: every input can interact with every other	tabular, small vectors	$n_{\text{in}} n_{\text{out}}$ per layer
Convolutional (CNN)	local patterns, the same pattern anywhere (translation equivariance)	images, spectra, time series	$C_{\text{out}}(C_{\text{in}}K + 1)$ per layer, independent of input size
Recurrent (RNN, LSTM)	order matters, state carries forward	sequences, kinetics	independent of sequence length
Attention (transformer)	every element may look at every other; the network decides which	text, sequences, sets, molecules	independent of sequence length; cost $\propto n^2$
Graph (GNN, message passing)	nodes interact along edges	molecules, crystals (SchNet, NequIP, MACE)	per edge type

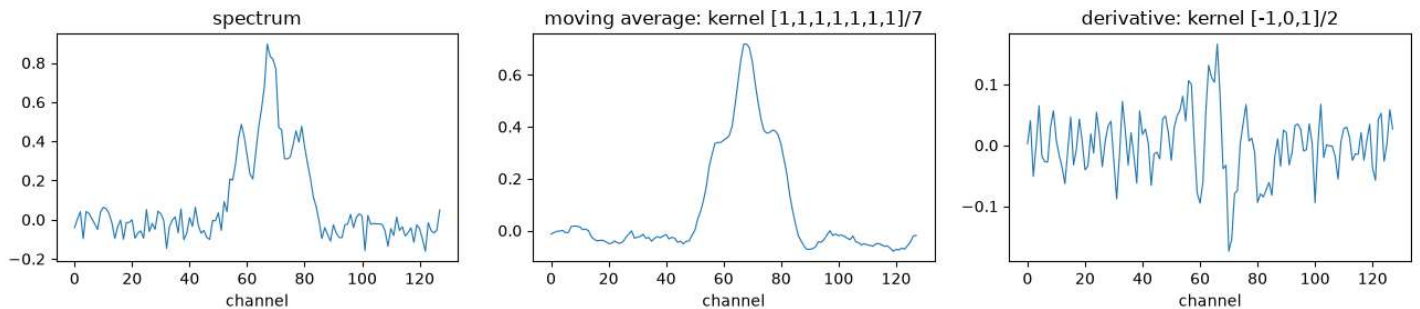
The less you build in, the more data you need. A transformer on 60 spectra loses to a CNN; on 10^9 sentences it wins.

Convolutional Neural Networks (CNNs)

- Convolution layer:** a kernel of K weights ($K \times K$ in 2-D) slides over the input; at each position the same weighted sum. Output = feature map
- Deep-learning "convolution" is a cross-correlation:

$$(x \star k)[i] = \sum_{j=0}^{K-1} x[i+j] k[j] \quad (I \star K)(i, j) = \sum_{m, n} I(i+m, j+n) K(m, n)$$

- **Channels:** 3 colour channels in, C_{out} feature maps out; each output channel has its own $C_{\text{in}} \times K \times K$ kernel
- **Padding** keeps the size (`padding='same'`); **stride** s divides it by s
- **Parameter count:** $C_{\text{out}}(C_{\text{in}}K^2 + 1)$. Lab 1: `Conv2D(32, 3)` on RGB has $32 \cdot (27 + 1) = 896$ parameters; the dense layer `Dense(128)` on 3072 inputs has 393 344
- **Receptive field:** how much of the input one output value sees; grows with every layer and every pooling step



Hand-made 1-D kernels on a spectrum

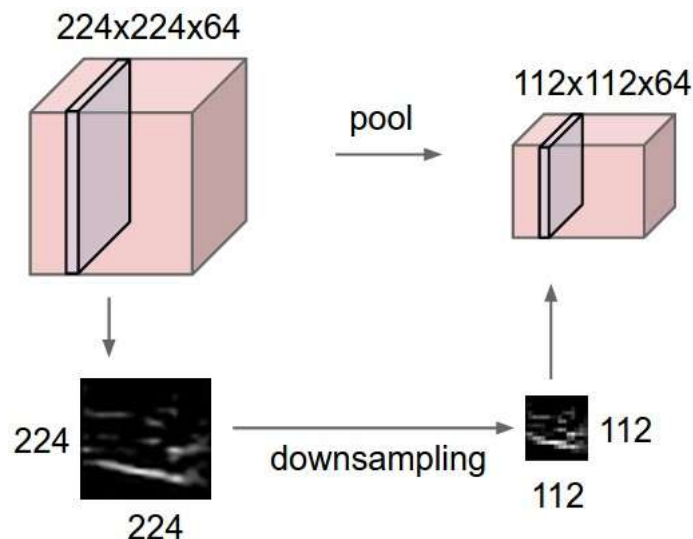
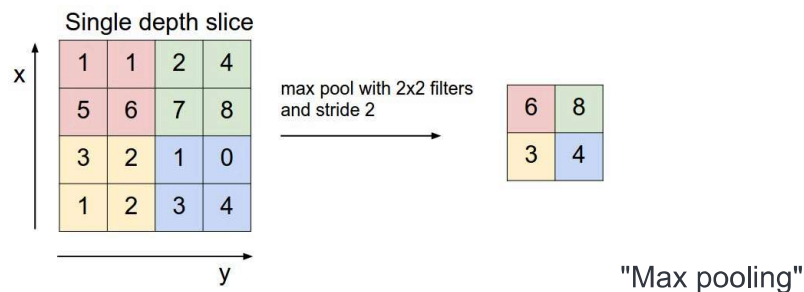


1711d181f1fd00c5c3fc61944c6974c2.png

Lab 2, Task 4.1: a smoothing kernel and a derivative kernel are convolutions you have used since Lab 1 of the course.

Pooling and the feature hierarchy

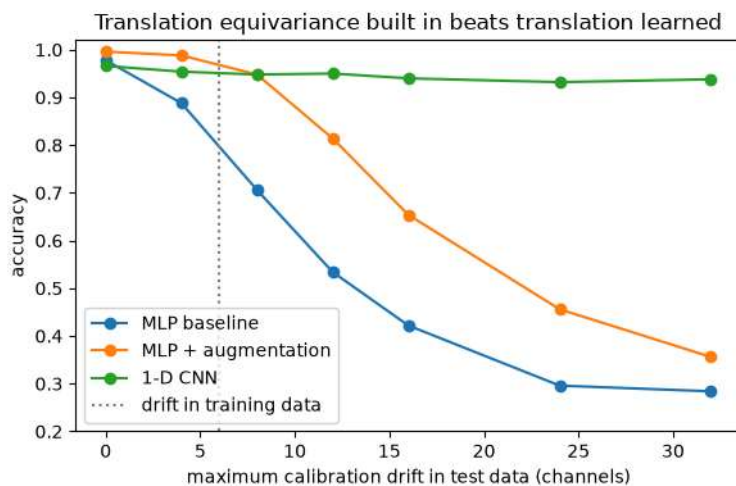
- **Max pooling** takes the maximum of each 2×2 block: halves the resolution, keeps the strongest response, makes the output insensitive to small shifts
- **Average pooling** takes the mean. **Global average pooling** averages each channel over the whole input: the classifier then only knows *whether* a feature occurred, not *where*
- Stacking convolution + pooling builds a hierarchy: edges $\rightarrow$ textures $\rightarrow$ parts $\rightarrow$ objects; in a spectrum: peaks $\rightarrow$ multiplets $\rightarrow$ compound
- Parameters stay small, receptive fields grow



35b1467ce12cb5489028c36c48fe6498.png

A 1-D CNN for spectra

- Spectra, chromatograms, decay traces and diffraction patterns are one-dimensional images. `Conv1d` slides a kernel along the channel axis
- Lab 2 network: `Conv1d(1, 8, 7) → pool → Conv1d(8, 16, 7) → pool → Conv1d(16, 16, 7) → global average pool → Linear(16, 4)` — 2852 parameters, receptive field 43 channels
- Trained on the same 60 spectra without augmentation, it beats the 10^5 -parameter MLP and stays accurate for calibration drifts five times larger than in training. Translation equivariance built in beats translation learned
- The learned first-layer kernels are smoothers, derivative filters and peak detectors — the same filters a spectroscopist would write by hand



Accuracy against calibration drift for MLP, augmented MLP and CNN

Lab 2, Tasks 4.2 and 4.3.

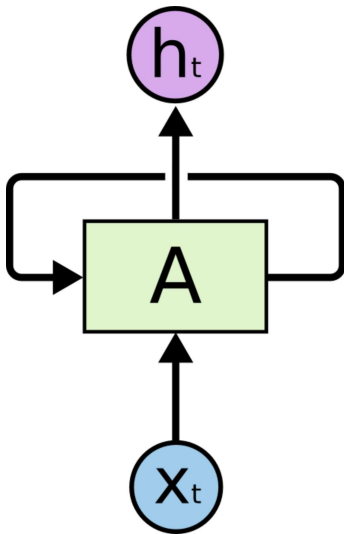
From LeNet to ResNet, and transfer learning

Year	Network	What it added
1998	LeNet-5	convolution + pooling + dense, digits (MNIST)
2012	AlexNet	GPUs, ReLU, dropout, data augmentation; won ImageNet by a wide margin
2014	VGG, Inception	depth (16–19 layers), parallel kernel sizes
2015	ResNet	skip connections, batch norm; 152 layers
2017	Transformer	attention replaces recurrence (next slides)
2020	Vision Transformer	image patches as tokens; needs 10^8 images to beat a CNN

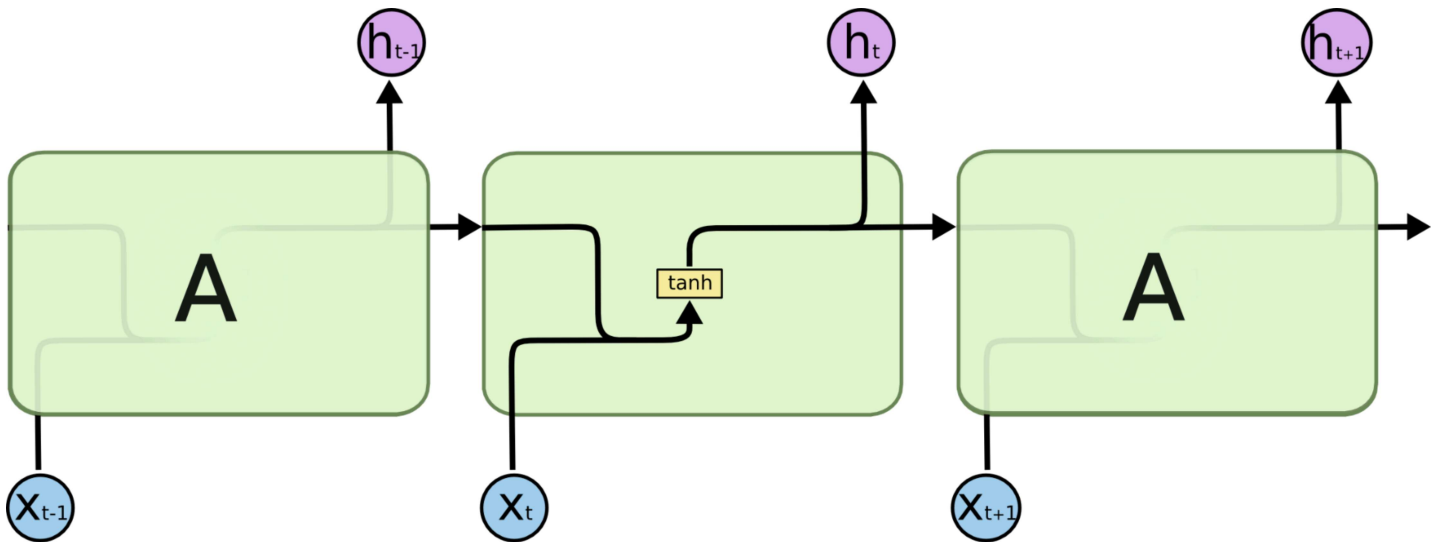
- **Transfer learning:** take a network trained on ImageNet, keep its convolution layers as a fixed feature extractor, train only a new classifier on your few hundred images. Works because early-layer features (edges, textures) are universal
- Lab 1, Part C: the bird/dog convolution layers as a feature extractor for 19 face images

Sequences I: recurrent networks and their bottleneck

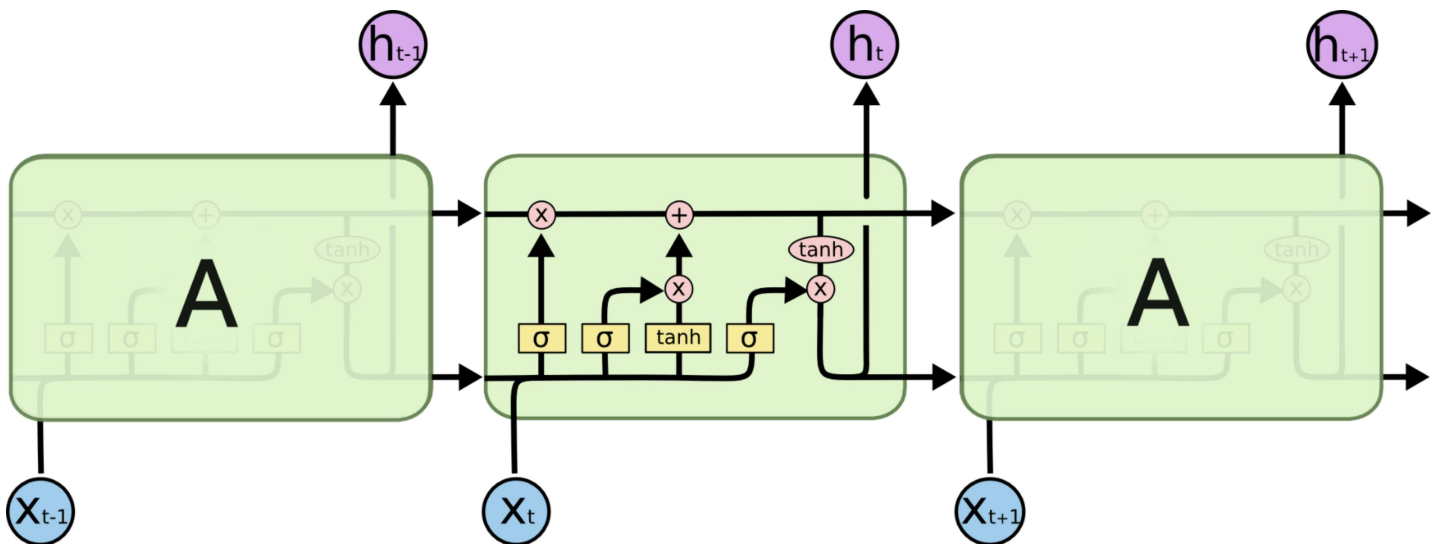
- A recurrent network reads a sequence one element at a time and carries a hidden state: $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b)$, $y_t = W_{hy}h_t + b_y$
- The same weights at every step: an RNN is a convolution in time with memory
- **LSTM** (Hochreiter & Schmidhuber 1997) adds a cell state C_t and three sigmoid gates (forget, input, output) that decide what to keep, write and read. It solved the vanishing gradient through time for sequences of a few hundred steps
- Two problems that attention solved: (1) all information about the past must pass through one fixed-size vector h_t ; (2) step t needs step $t - 1$: no parallelism, slow training
- RNNs and LSTMs remain useful for kinetics, sensor streams and small sequence problems. Large language models do **not** use them: ChatGPT and its relatives are transformers



1304196c7fb66cf863bd862a136354ea.png



94f9c71406139bfc9e4446c8a51050a9.png



3e77f1d945304bbfed9456dbb111329d.png

Sequences II: Attention or the attempt to repair RNN by giving it long term memory and choose

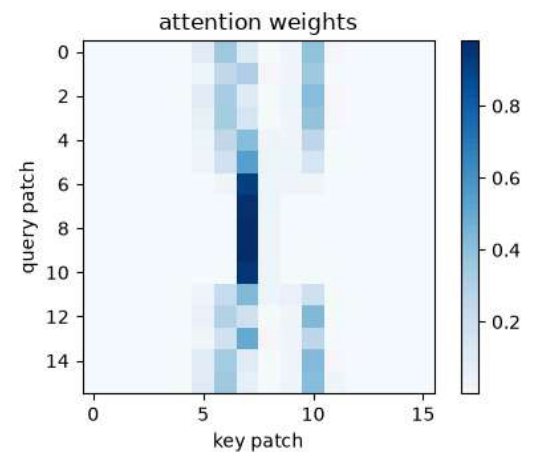
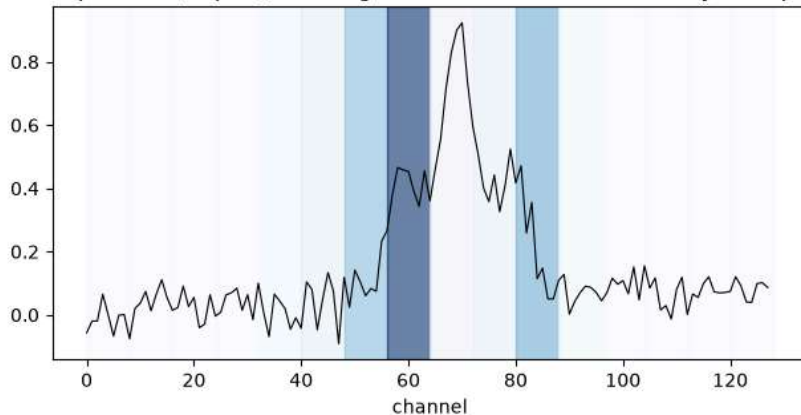
RNN (historic) $\rightarrow$ RNN with attention (see all input) $\rightarrow$ Transformer = attention without the RNN

Given n queries Q (what am I looking for), m keys K (dimension d) (how do i identify myself) and m values V (what do I give back if taken):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

- Row i of $QK^\top$: the similarity of query i with every key. Softmax makes the weights positive and normalized. The output is the weighted average of the values
- $\sqrt{d}$ keeps the dot products of order one so the softmax does not saturate
- **Self-attention:** $Q = XW_Q$, $K = XW_K$, $V = XW_V$ all from the same input X ; every element looks at every other element and decides, from the data, how much
- or:
 - The transformer compares your query with all keys.
 - The best-matching keys receive the largest attention weights.
- In numpy: three lines. `A = softmax(Q @ K.T / np.sqrt(d)); out = A @ V`
- Cost $\propto n^2d$: the price for letting everything see everything

test spectrum (triplet); shading = mean attention received by each patch



Lab 2, Task 5.1.

Attention as a soft library lookup

- Keys = the reference spectra of a library, values = their labels, query = the measured spectrum
- $QK^\top$ = overlap of the measurement with each reference; softmax = normalized, differentiable "closest match"; output = class probabilities
- A dictionary lookup that you can back-propagate through. Every transformer is a stack of such lookups where keys, values and queries are all learned
- **Multi-head attention:** h independent sets of (W_Q, W_K, W_V) with dimension d/h each, outputs concatenated. Each head learns a different kind of relation (position, chemistry, syntax)
- **Positional encoding:** attention treats its input as a set; permute the tokens and the outputs permute with them. Add a position-dependent vector (sinusoids in 2017, learned or rotary today) so the network

can tell where a token is

Lab 2, Parts 5.2 and 5.3.

The transformer block

$x \rightarrow \text{LayerNorm} \rightarrow \text{Multi-head self-attention} \rightarrow + x \text{ (residual)}$
 $\rightarrow \text{LayerNorm} \rightarrow \text{MLP (Linear, GELU, Linear)} \rightarrow + x \text{ (residual)}$

- Stack N blocks ($N = 6$ in 2017, $N \approx 100$ in current language models). Every ingredient of this lecture is inside: Adam with warm-up, layer normalization, residual connections, dropout, weight decay
- **Encoder**: all tokens see all tokens (BERT, vision transformers, protein language models)
- **Decoder**: a causal mask sets $A_{ij} = 0$ for $j > i$, so a token sees only its past. Trained to predict the next token; generates by sampling one token at a time (GPT, ChatGPT, Claude)
- Why it replaced RNNs: no bottleneck (every token can reach every other in one step), fully parallel training, and it keeps improving with data and size where LSTMs saturated
- Vaswani et al. 2017, "Attention is all you need": <https://arxiv.org/abs/1706.03762>

Lab 2, Task 5.2: a one-block, one-head transformer on patches of a spectrum.

Attention in chemistry and physics

- **AlphaFold 2** (2021): attention over residue pairs; the "Evoformer" exchanges information between the sequence alignment and the pair-distance representation. Structure prediction at experimental accuracy
 - **Machine-learned force fields**: SchNet (continuous convolutions over neighbours), NequIP and MACE (equivariant message passing). Graph networks whose "attention" is restricted to atoms within a cut-off, with rotational symmetry built in — inductive bias again
 - **Spectrum to structure**: transformers that read an IR, NMR or mass spectrum and write a SMILES string; encoders pre-trained on millions of computed spectra
 - **Language models for chemistry**: ChemBERTa, MolGPT; and the general-purpose LLM as a coding assistant for your data analysis
 - **Transformers for time series and detectors**: trigger decisions at CERN, denoising of cryo-EM images
 - Common pattern: pre-train a large model on cheap data (simulations, unlabeled spectra), fine-tune on the few hundred measurements you have
-

Generative models in one table

Model	Idea	Trained by	Used for
Autoencoder (AE)	encoder $\rightarrow$ bottleneck of size $k \rightarrow$ decoder	reconstruction error	compression, denoising; linear AE = rank- k SVD (Lab 2, Part 6)
Variational AE (VAE)	bottleneck is a distribution; sample from it	reconstruction + KL term	smooth latent spaces, molecule generation
Generative adversarial network (GAN)	generator makes fakes, discriminator tells real from fake; trained against each other	minimax game	realistic images; prone to non-convergence and mode collapse
Diffusion model	learn to remove noise step by step; generate by denoising pure noise	denoising regression	image generation (Stable Diffusion), molecular conformers, protein design
Autoregressive LLM	predict the next token	cross-entropy on text	ChatGPT, code assistants; "reasoning" models generate and score many candidate chains of thought

- The autoencoder closes the loop with the unsupervised lecture: bottleneck k = number of species; nonlinear decoder = a curved low-dimensional manifold
- GANs and diffusion models can also **augment** small experimental data sets, but validate on real data only

Lecture 2 — Summary: three levers

1. **The optimizer:** mini-batches for cheap noisy gradients, momentum to smooth, Adam to rescale per parameter, a schedule to settle. Learning rate first, always
2. **Regularization:** early stopping, weight decay, dropout limit memorization; augmentation adds knowledge. Watch the two loss curves
3. **Architecture:** build in what you know (convolution = local and translation-equivariant; graph = bonds; attention = let the data decide). Less bias, more data

And one diagnostic habit: check the loss at initialization, overfit a small batch, scan the learning rate.

Further reading

- Bishop & Bishop, *Deep Learning: Foundations and Concepts* (Springer 2024), free at <https://www.bishopbook.com> — the mathematically careful version of these two lectures
 - Prince, *Understanding Deep Learning* (MIT Press 2023), free at <https://udlbook.github.io/udlbook/> — best figures; chapters 6 (fitting), 9 (regularization), 10 (CNN), 12 (transformers)
 - Goodfellow, Bengio & Courville, *Deep Learning* (MIT Press 2016), free at <https://www.deeplearningbook.org> — chapters 8 (optimization), 7 (regularization), 9 (CNN)
 - Vaswani et al., *Attention is all you need* (2017), <https://arxiv.org/abs/1706.03762>
 - Kingma & Ba, *Adam* (2015), <https://arxiv.org/abs/1412.6980>; Loshchilov & Hutter, *Decoupled weight decay* (AdamW, 2019), <https://arxiv.org/abs/1711.05101>
 - He et al., *Deep residual learning* (2015), <https://arxiv.org/abs/1512.03385>
 - Jay Alammar, *The Illustrated Transformer*, <https://jalammar.github.io/illustrated-transformer/>
 - Andrej Karpathy, *Let's build GPT from scratch* (video, 2 h), <https://www.youtube.com/watch?v=kCc8FmEb1nY>, and *A recipe for training neural networks*, <https://karpathy.github.io/2019/04/25/recipe/>
 - 3Blue1Brown, *Attention in transformers, visually explained*, <https://www.3blue1brown.com/lessons/attention>
 - Jumper et al., *Highly accurate protein structure prediction with AlphaFold* (Nature 2021), <https://doi.org/10.1038/s41586-021-03819-2>
 - Batatia et al., *MACE* (2022), <https://arxiv.org/abs/2206.07697>
-

Learning questions

1. Write down one training step (forward, loss, backward, update). Which of the four lines does each of the following change: momentum, dropout, weight decay, a learning-rate schedule, augmentation?
2. Why can curve fitting use Gauss–Newton or Levenberg–Marquardt, and why can a network with 10^6 parameters not? What do momentum and RMSProp try to recover?
3. Explain mini-batch SGD, momentum, RMSProp and Adam. Why does Adam tolerate a larger learning rate?
4. What happens when the learning rate is too small, and when it is too large? How do you find a good one?
5. Why does initialization matter? State the Xavier and He rules and say when each applies
6. Where do vanishing gradients come from, and which three architectural cures are in daily use?
7. Name four regularization methods and say for each whether it limits the model or informs it. Which one helped most on drifting spectra, and why?
8. Explain dropout. Why must it be switched off at test time?

9. What is the difference between grid search, random search and Bayesian optimization? Which set do you tune on?
10. Compute the number of parameters of a convolution layer with 8 input channels, 16 output channels and kernel length 7. Why does it not depend on the length of the spectrum?
11. Why do we stack convolution and pooling? What does global average pooling throw away, and why is that useful for a drifting spectrometer?
12. What is the inductive bias of a CNN, of an RNN, of a transformer? Which needs the most data, and why?
13. Write down the attention formula and explain the roles of Q , K and V with the spectral-library picture. What does the $\sqrt{d}$ do?
14. Why does a transformer need a positional encoding? What is a causal mask for?
15. What were the two problems of RNNs that attention solved?
16. Explain the relation between a linear autoencoder with bottleneck k and a rank- k SVD. When does a nonlinear autoencoder do better?
17. You train a network and the loss does not move. List five checks, in order
18. Name three uses of neural networks in chemistry or physics and say which architecture each uses and why

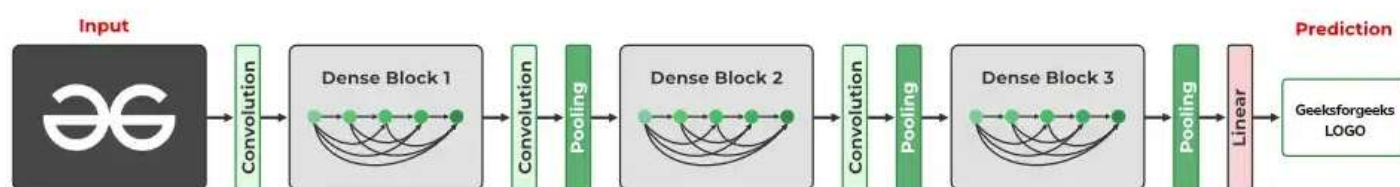
Appendix: a zoo of network types (not complete)

Type	Purpose
Feed-forward network (MLP), perceptron	The basic building block: dense layers, no cycles
Convolutional network (CNN)	Local, translation-equivariant features in images, spectra, time series
Residual network (ResNet)	CNN with skip connections; makes 100+ layers trainable
U-Net	Encoder–decoder CNN with skip connections at each resolution; image segmentation, denoising
DenseNet, Highway, Inception	Variants of skip connections and parallel kernel sizes
Recurrent network (RNN), LSTM, GRU	Sequences with a carried state; gates control memory
Bidirectional RNN	Reads the sequence forwards and backwards
Transformer (encoder, decoder)	Self-attention; language, vision, proteins
Graph network (GNN, message passing; SchNet, NequIP, MACE)	Nodes exchange messages along edges; molecules, crystals

Type	Purpose
Autoencoder (AE), denoising AE, variational AE	Compression to a bottleneck; generation from the latent space
Generative adversarial network (GAN)	Generator against discriminator
Diffusion model	Iterative denoising; state of the art in image and molecule generation
Normalizing flow	Invertible network; exact likelihoods
Radial basis function network	Sum of Gaussians with learned centres; the classical function approximator
Hopfield network, Boltzmann machine, deep belief network	Energy-based models; historically important, rarely used today
Kohonen map (SOM)	Unsupervised 2-D map of the data; competitive learning
Echo state / liquid state machine	Fixed random recurrent reservoir, only the read-out is trained
Neural ODE	The layer index becomes continuous time; the network is a differential equation
Physics-informed network (PINN)	Loss includes the residual of a differential equation

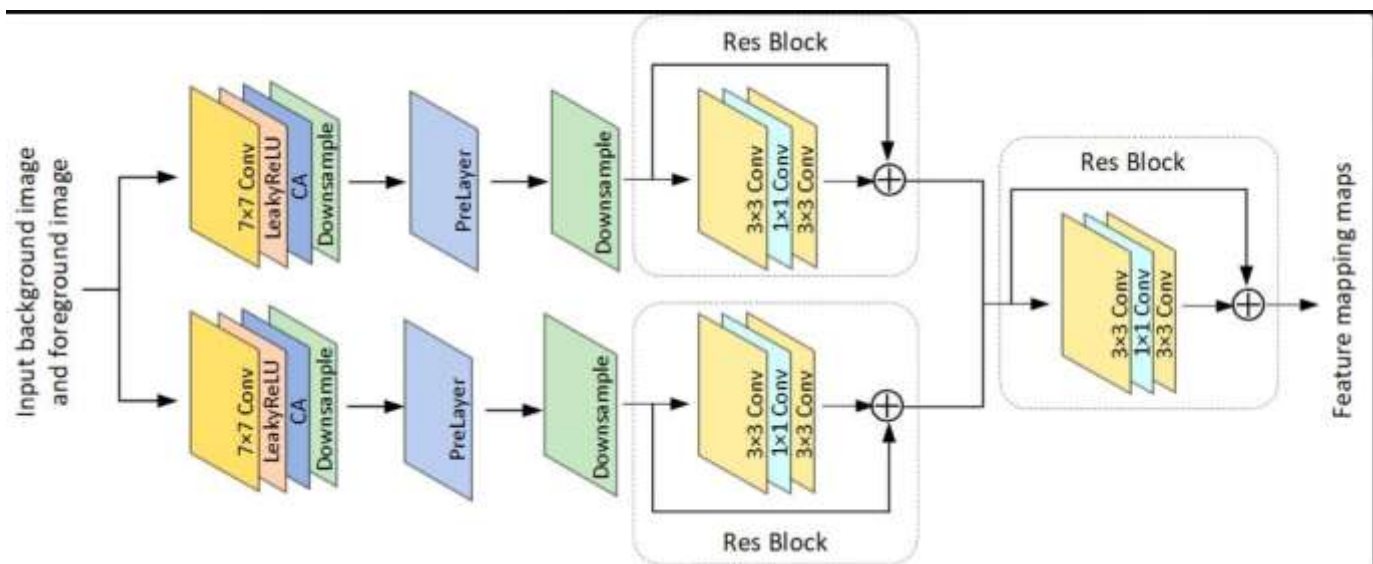
Appendix: block types that improve gradient flow

DenseNet: every layer receives all earlier layers



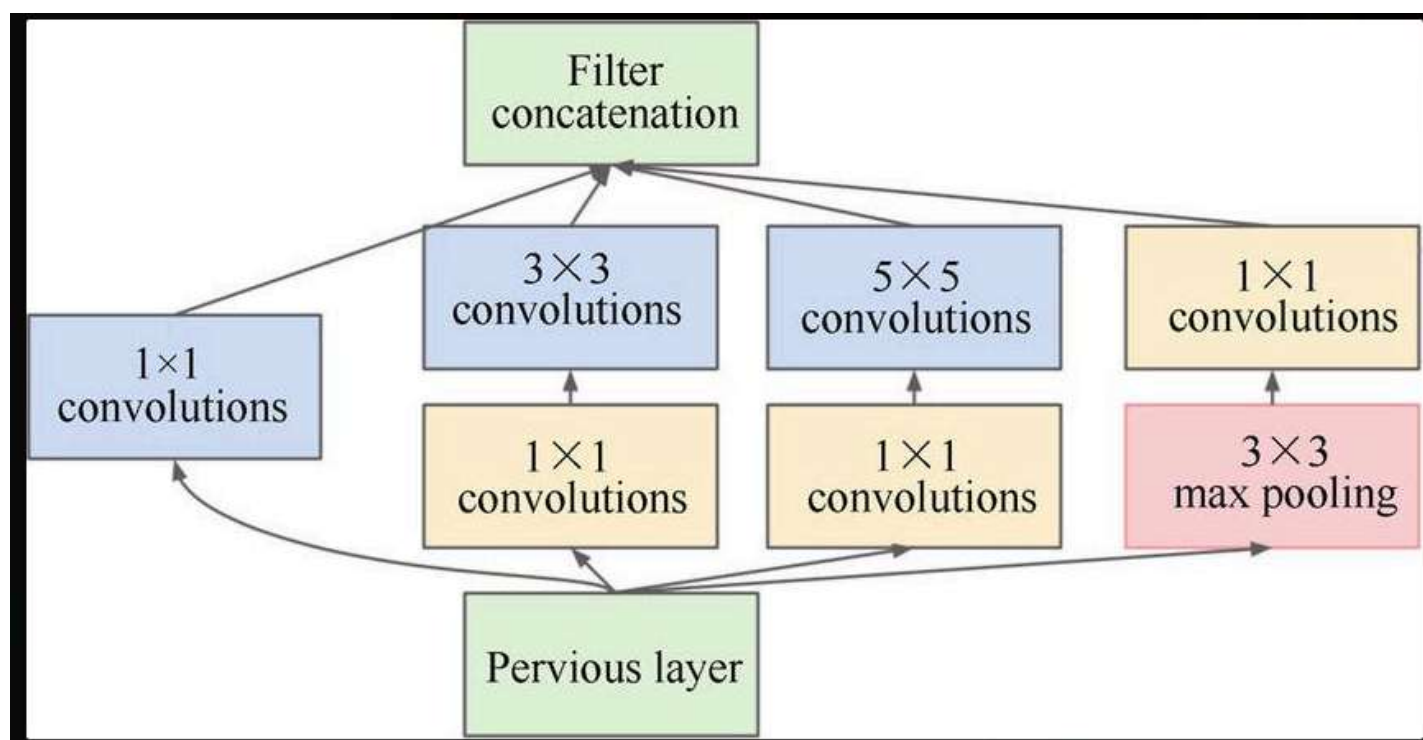
b4dc0f86d5ff2762cf366690e76c360b.png

Highway networks: gated skip connections



7dbf70da930a35d4b1d6b7bfa69ea5b0.png

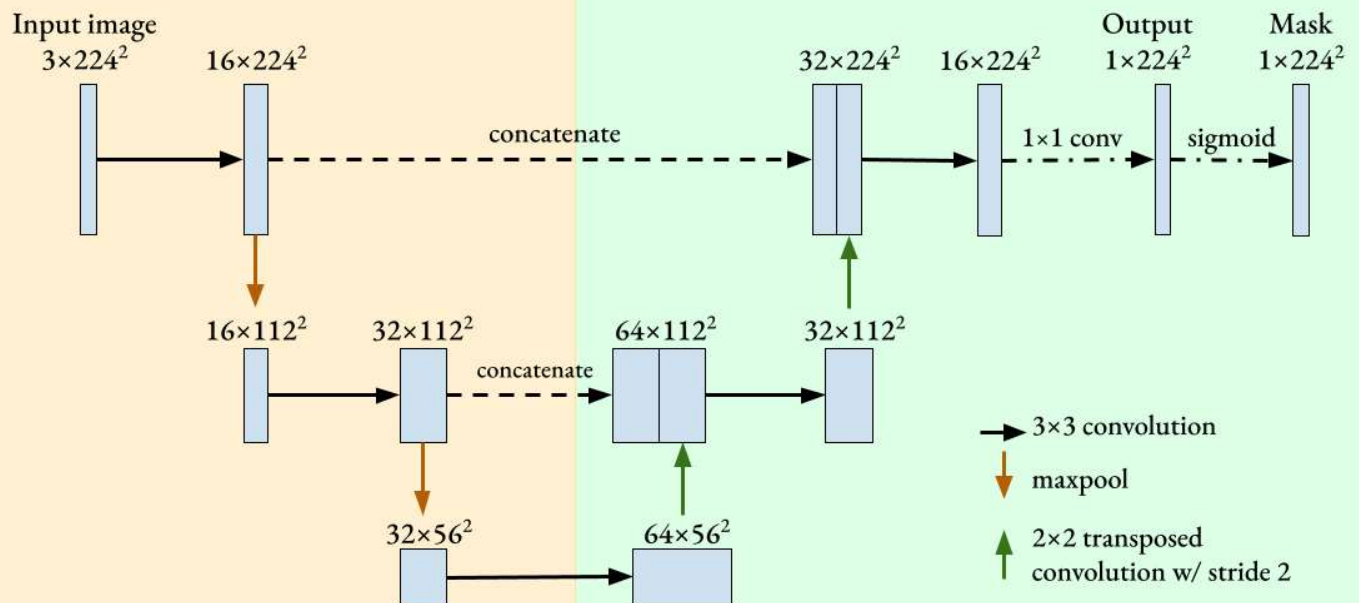
Inception: parallel kernel sizes in one block



acaa29d6fa7ff0ffe97d6757e1289775.png

U-Net: encoder–decoder with skip connections at each resolution

Encoder-Decoder with skip connections (U-Net)

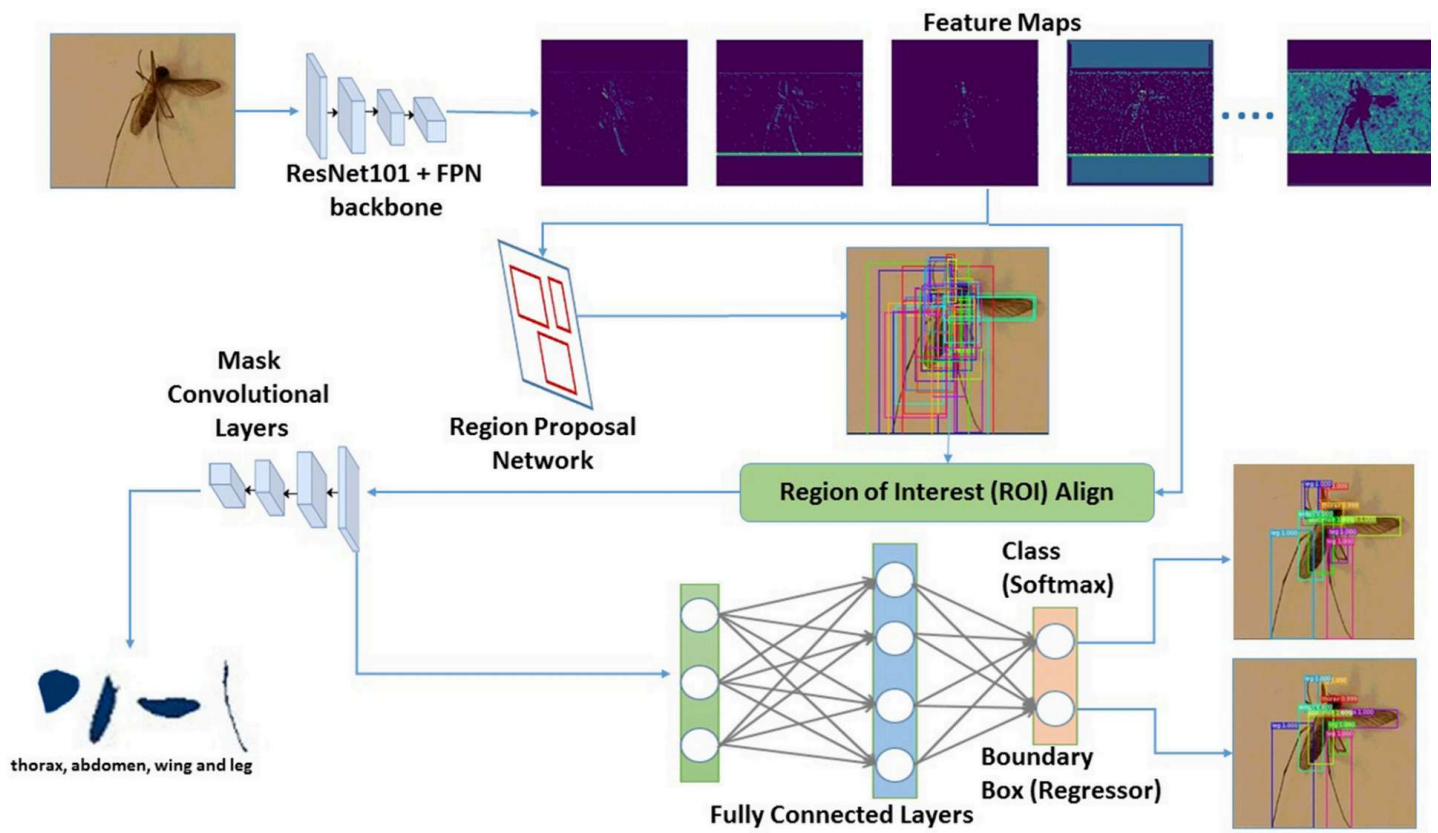


This part extracts relevant features from image

This part builds segmentation mask from features

8b50b682567747b5332cb208a28db5e3.png

Masked attention: a token sees only its past



67a208d8af004764f82badcbba9607ea.png